

Final Project - Computer Vision

Yibo Zhou

Date: September, 2025



Department of Electrical Engineering

EEN020 - Computer Vision

Abstract

This project investigates the classical Structure-from-Motion (SfM) pipeline on nine image datasets. The workflow consists of SIFT feature extraction and matching, outlier rejection using RANSAC, and camera motion estimation via essential matrix and homography decomposition. Experimental results demonstrate that the algorithm can robustly reconstruct the majority of scenes. The generated point clouds capture clear structural details, and the estimated camera poses align well with the actual viewpoints. However, in several datasets, the reconstruction failed due to challenges such as uneven feature distribution and limited camera baselines. These results confirm the effectiveness of the classical SfM pipeline under favorable conditions, while also highlighting its limitations in more complex or degenerate scenarios.

(1) Calculate relative orientations

(1.1) SIFT feature extraction

In this final project, SFM (Structure From Motion) algorithm is used to reconstruct 3D scene from several 2D images.

SIFT (Scale-Invariant Feature Transform) is firstly been used. SIFT is a computer vision algorithm used to detect and describe local features in images. SIFT features have properties include: scale Invariance, rotation invariance, illumination and affine invariance and distinctive descriptors. Thus, SIFT feature extractions and feature mappings across different views are necessary to the following steps.

To implement SIFT algorithm efficiently in MATLAB, VLFeat (Vision Lab Features Library) is used in this project.

Code below shows how to extract SIFT features **f** and corresponding descriptors **d** from image **img** by VLFeat toolbox.

```
[f, d] = vl_sift(single(rgb2gray(img)), 'PeakThresh', 1);
```

(1.2) Feature mapping

In order to know the position of the same 3D point in different images, feature mapping is used to match the extracted SIFT features according to their descriptors.

Code below shows how Approximate Nearest Neighbor (ANN) algorithm is used to match SIFT descriptors **d1** and **d2** by VLFeat toolbox, where the output **matches** are the matched feature indices.

```
[matches, ~] = vl_ubcmatch(d1, d2);
```

Then, matched features **x1** and **x2** are extracted according to **matches**, which is shown below.

```
x1 = [f1(1, matches(1, :)); f1(2, matches(1, :))];
```

```
x2 = [f2(1, matches(2, :)); f2(2, matches(2, :))];
```

(1.3) Calculate relative orientations by essential matrix estimation (8-point algorithm)

For 3D scenes with strong depth changes, 8-point algorithm performs better to calculate relative orientations ($R_{i,i+1}|T_{i,i+1}$) by estimating the essential matrix E .

Firstly, matched features $\mathbf{x}$ and $\bar{\mathbf{x}}$ are normalized by camera calibration matrices K and $\bar{K}$:

$$\mathbf{y}_i = K\mathbf{x}_i \quad (1)$$

$$\bar{\mathbf{y}}_i = \bar{K}\bar{\mathbf{x}}_i \quad (2)$$

According to the epipolar constraint, we have:

$$\bar{\mathbf{y}}_i E \mathbf{y}_i = 0 \quad (3)$$

Therefore each correspondence gives one linear constraint on the entries of E . In matrix form we can write the resulting system as

$$\begin{bmatrix} \bar{x}_1 x_1 & \bar{x}_1 y_1 & \bar{x}_1 z_1 & \bar{y}_1 x_1 & \bar{y}_1 y_1 & \bar{y}_1 z_1 & \bar{z}_1 x_1 & \bar{z}_1 y_1 & \bar{z}_1 z_1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \bar{x}_i x_i & \bar{x}_i y_i & \bar{x}_i z_i & \bar{y}_i x_i & \bar{y}_i y_i & \bar{y}_i z_i & \bar{z}_i x_i & \bar{z}_i y_i & \bar{z}_i z_i \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ \bar{x}_n x_n & \bar{x}_n y_n & \bar{x}_n z_n & \bar{y}_n x_n & \bar{y}_n y_n & \bar{y}_n z_n & \bar{z}_n x_n & \bar{z}_n y_n & \bar{z}_n z_n \end{bmatrix} \begin{bmatrix} E_{11} \\ E_{12} \\ E_{13} \\ E_{21} \\ E_{22} \\ E_{23} \\ E_{31} \\ E_{32} \\ E_{33} \end{bmatrix} = 0 \quad (4)$$

Thus, we have formed the matrix for DLT:

$$\min_{\|v\|^2=1} \|Mv\|^2 \quad (5)$$

whose solution v is the last column of V_M from

$$M = U_M S_M V_M^\top \quad (6)$$

The estimated essential matrix E can then be extracted from v .

To compute camera rotations and translations from E , SVD is applied, where singular values are ensured to be 0, 0 and 1:

$$E = U_E \text{diag}\left(\begin{bmatrix} 1 & 1 & 0 \end{bmatrix}\right) V_E^\top \quad (7)$$

Therefore, obtained solutions are:

$$P_1 = \begin{bmatrix} I & 0 \end{bmatrix}$$

$$P_2 = \begin{bmatrix} U_E W V_E^\top & u_3 \end{bmatrix} \text{ or } \begin{bmatrix} U_E W^\top V_E^\top & u_3 \end{bmatrix} \text{ or } \begin{bmatrix} U_E W V_E^\top & -u_3 \end{bmatrix} \text{ or } \begin{bmatrix} U_E W^\top V_E^\top & -u_3 \end{bmatrix}$$

$$\text{where } u_3 \text{ is the third column of } U_E \text{ and } W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

(1.4) Calculate relative orientations by homography estimation

For 3D scenes with a dominant plane, homography performs better to calculate relative orientations $(R_{i,i+1}|T_{i,i+1})$ by estimating the invertible matrix H .

According to the homography theory, we have:

$$\mathbf{y} \sim H\bar{\mathbf{y}} \quad (8)$$

where $\mathbf{y}$ and $\bar{\mathbf{y}}$ are the normalized matched features.

According to relation $\lambda\mathbf{y} = H\bar{\mathbf{y}}$, each camera contributes 3 equations and 1 extra unknown λ . Thus, letting equations be more than the unknowns, we need $3n \geq 8 + n \Leftrightarrow n \geq 4$, at least 4 image points to estimate H . In matrix form, we can construct the equation:

$$\begin{bmatrix} \bar{\mathbf{y}}^\top & 0 & 0 & -\mathbf{y}_1 \\ 0 & \bar{\mathbf{y}}^\top & 0 & -\mathbf{y}_2 \\ 0 & 0 & \bar{\mathbf{y}}^\top & -\mathbf{y}_3 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ \lambda \end{bmatrix} = 0 \quad (9)$$

$$\text{where } H = \begin{bmatrix} h_1^\top \\ h_2^\top \\ h_3^\top \end{bmatrix} \text{ and } \mathbf{y} = \begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \\ \mathbf{y}_3 \end{bmatrix}.$$

To use the DLT algorithm, we can formulate the equations as follows:

$$\min_{\|v\|^2} \|Mv\|^2 \quad (10)$$

where

$$M = \begin{bmatrix} \bar{\mathbf{y}}_1^\top & 0 & 0 & -\mathbf{y}_{11} & 0 & \dots & 0 & 0 \\ 0 & \bar{\mathbf{y}}_1^\top & 0 & -\mathbf{y}_{12} & 0 & \dots & 0 & 0 \\ 0 & 0 & \bar{\mathbf{y}}_1^\top & -\mathbf{y}_{13} & 0 & \dots & 0 & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots & \vdots \\ \bar{\mathbf{y}}_i^\top & 0 & 0 & 0 & 0 & \dots & 0 & -\mathbf{y}_{i1} \\ 0 & \bar{\mathbf{y}}_i^\top & 0 & 0 & 0 & \dots & 0 & -\mathbf{y}_{i2} \\ 0 & 0 & \bar{\mathbf{y}}_i^\top & 0 & 0 & \dots & 0 & -\mathbf{y}_{i3} \end{bmatrix} \quad (11)$$

$$v = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \\ \lambda_1 \\ \vdots \\ \lambda_i \end{bmatrix} \quad (12)$$

$$\mathbf{y}_i = \begin{bmatrix} \mathbf{y}_{i1} \\ \mathbf{y}_{i2} \\ \mathbf{y}_{i3} \end{bmatrix} \quad (13)$$

The solution v is the last column of V_M from $M = U_M S_M V_M^\top$, and H can be extracted from its first nine components.

Code below shows how to extract the camera matrix from H :

$$[\mathbf{R1}, \mathbf{t1}, \sim, \mathbf{R2}, \mathbf{t2}, \sim, \sim] = \text{homo_to_Rt}(H);$$

Therefore, obtained solutions are:

$$P_1 = \begin{bmatrix} I & 0 \end{bmatrix}$$

$$P_2 = \begin{bmatrix} R_1 | t_1 \end{bmatrix} \text{ or } P_2 = \begin{bmatrix} R_2 | t_2 \end{bmatrix}$$

(1.5) Estimation Comparison

(1.5.1) Select P_2 with where points are in front of both cameras

There are several candidate P_2 estimations obtained from the essential matrix and homography, but only one from each method has all scene points in front of both cameras. To select the one, we need to reconstruct scene points $\mathbf{X}$ from the known $\mathbf{x}$, P_1 and P_2 .

According to relation $P\mathbf{X} = \lambda\mathbf{x}$, each camera contributes 3 equations and 1 extra unknown λ . Thus, to let equations being more than the unknowns, we need $3n \geq 3 + n \Leftrightarrow n \geq \frac{3}{2}$, at least 2 cameras to estimate the scene point. In matrix form, we can construct the equation:

$$\begin{bmatrix} P_1 & \mathbf{x}_1 & 0 \\ P_2 & 0 & \mathbf{x}_2 \end{bmatrix} \begin{bmatrix} \mathbf{X} \\ -\lambda_1 \\ -\lambda_2 \end{bmatrix} = 0 \quad (14)$$

To use the DLT algorithm, we can formulate the equations as follows:

$$\min_{||v||^2} ||Mv||^2 \quad (15)$$

where

$$M = \begin{bmatrix} P_1 & \mathbf{x}_1 & 0 \\ P_2 & 0 & \mathbf{x}_2 \end{bmatrix} \quad (16)$$

$$v = \begin{bmatrix} \mathbf{X} \\ -\lambda_1 \\ -\lambda_2 \end{bmatrix} \quad (17)$$

The solution v is the last column of V_M from $M = U_M S_M V_M^\top$, and the reconstructed scene point $\mathbf{X}$ is its first four components.

Besides, according to the formulae, camera centre C is the null space of P , and its viewing direction $\vec{d}$ is the first three elements of P 's third row.

Overall, we can determine whether the scene point is in front of the camera by the sign of the vector dot product $C\vec{\mathbf{X}} \cdot \vec{d}$. The candidate P_2 with the most points in front of the camera can be considered the correct one.

(1.5.2) Use RANSAC to estimate robustly

While estimating camera matrices using essential matrix and homography matrix, since point correspondences are determined automatically using SIFT descriptors, outliers have to be removed as there are a small portion of incorrect matches.

RANSAC (Random sample consensus) is used to deal with this problem and to make estimations more robust. $n = 2$ matches in the estimation of the essential matrix and $n = 4$ matches in the estimation of the homography are randomly selected from all SIFT feature matches. Solutions are evaluated by error residuals for the rest of the measurements:

For essential matrix solution:

$$\text{error}_i = \frac{1}{2} [d^2(\bar{\mathbf{y}}_i, E\mathbf{y}_i) + d^2(\mathbf{y}_i, E^\top \bar{\mathbf{y}}_i)] \quad (18)$$

$$d(\mathbf{p}, \mathbf{l}) = \frac{|ax + by + c|}{\sqrt{a^2 + b^2}}, \text{ where } \mathbf{p} = \begin{bmatrix} x & y & 1 \end{bmatrix}^\top \text{ and } \mathbf{l} = \begin{bmatrix} a & b & c \end{bmatrix}^\top \quad (19)$$

For homography solution:

$$\text{error}_i = \frac{1}{2} [(H\bar{\mathbf{y}}_i - \mathbf{y}_i)^2 + (H^{-1}\mathbf{y} - \bar{\mathbf{y}})^2] \quad (20)$$

Points whose error residuals are less than a predefined error threshold can be regarded as inliers.

This process is repeated for a number of times until the inlier number reach a pre-defined inlier threshold. E and H that reach their thresholds are regarded as the final solution.

(1.5.3) Result comparison between two methods

The reason why we estimate P_2 respectively with E and H in parallel, is because they can handle different scenes. For the essential matrix, it can reconstruct scenes with 3D structures, but cannot handle the noise very well if all points are on a dominant plane. For the homography, it can express the relation between 2 surfaces, but the estimation can be disturbed by noises if the scene has complex 3D structure.

To choose the best P_2 from the alternatives of two means, we can reconstruct the scene respectively, and compare the number of inliers that being in front of both cameras. The one with better performance is chosen to be the final P_2 result.

(2) Upgrade to absolute rotations

In section 1, for each 2 images, we have estimated relative rotations $R_{(i-1,i)}$ consecutively. To upgrade absolute rotations for all cameras, we can set $R_1 = I$, and all other rotation matrices can then be upgraded by:

$$R_i = R_{(i-1,i)} R_{i-1} \quad (21)$$

(3) Initial reconstruction

To estimate translations in following steps, for each dataset, two initial images are suggested to reconstruct initial 3D points.

Firstly, as discussed in Section 1.1 and Section 1.2, we extract SIFT features of the two initial images, and map them together according to their descriptors.

Next, as discussed in Section 1.2, Section 1.3, essential matrix method and homography method are used in parallel to better estimate their relative position P_{i_2,i_1} .

Then, as shown in (14)-(17), initial scene points $\mathbf{X}_0$ can be reconstructed by $\mathbf{x}_{i_1}$, $\mathbf{x}_{i_2}$ and P_{i_2,i_1} .

Specifically, as discussed in Section 1.5.2 and Section 1.5.3, RANSAC is used to better reconstruct scene points. Residuals are used to evaluate the estimation, and the $\mathbf{X}_0$ with the most inliers is saved as the final chosen initial reconstruction.

Finally, since the first camera is set at the orientation during the estimation, $\mathbf{X}_0$ under the relative coordinate need to be rotated to the world coordinate:

$$\mathbf{X}_0^{world} = R_{i_1}^\top \mathbf{X}_0^{relative} \quad (22)$$

(4) Estimate translations T_i robustly

In Section 1, relative translations are estimated consecutively with relative rotations. However, since translation scale λ is unknown, chaining translations causes drifts. Thus, better translations is estimated based on absolute rotations R_i and fixed scene points $\mathbf{X}_0$, which is accomplished in Section 2 and Section 3.

Code below shows how VLFeat toolbox is firstly used to match 2D image points and 3D scene points according to their SIFT descriptors `sift_ds{i}` and `desc_X`, where the output `matches` are the matched feature indices.

$$[\text{matches}, \sim] = \text{vl_ubcmatch}(\text{sift_ds}\{\mathbf{i}\}, \text{desc_X});$$

For the i th camera, according to relation $\lambda_j \mathbf{x}_j = R_i \mathbf{X}_j + T_i$, each match $\mathbf{x}_j - \mathbf{X}_j$ contributes 3 equations and 1 extra unknown λ_j . Thus, letting equations be more than the unknowns, we need $3n \geq 3 + n \Leftrightarrow n \geq 1.5$, at least 2 matches to estimate T_i . In matrix form, we can construct the equation:

$$\begin{bmatrix} I & \begin{bmatrix} \mathbf{x} \\ 1 \end{bmatrix} & R_i \mathbf{X} \end{bmatrix} = \begin{bmatrix} sT_i \\ -s\lambda \\ s \end{bmatrix} \quad (23)$$

where s is a scalar for the matrix form construction.

To use the DLT algorithm, we can formulate the equations as follows:

$$\min_{\|v\|^2} \|Mv\|^2 \quad (24)$$

where

$$M = \begin{bmatrix} I & \begin{bmatrix} \mathbf{x}_1 \\ 1 \end{bmatrix} & 0 & R_i \mathbf{X}_1 \\ I & 0 & \begin{bmatrix} \mathbf{x}_2 \\ 1 \end{bmatrix} & R_i \mathbf{X}_2 \end{bmatrix} \quad (25)$$

$$v = \begin{bmatrix} sT_i \\ -s\lambda_1 \\ -s\lambda_2 \\ s \end{bmatrix} \quad (26)$$

The solution v is the last column of V_M from $M = U_M S_M V_M^\top$, and T_i can be extracted from its first 3 components and last component.

As explained in Section 1.5.2, RANSAC is used here to make estimations more robust. $n = 2$ matches are randomly selected from all $\mathbf{x}_j - \mathbf{X}_j$ matches. Solutions are evaluated by error residuals for the rest of the measurements:

$$\text{error}_j = \|\mathbf{x}_j - \text{projection}_j\|^2 \quad (27)$$

$$\text{projection}_j = \begin{bmatrix} \frac{R_{j,1}^\top \mathbf{x}_j + T_{j,1}}{R_{j,3}^\top \mathbf{x}_j + T_{j,3}} & \frac{R_{j,2}^\top \mathbf{x}_j + T_{j,2}}{R_{j,3}^\top \mathbf{x}_j + T_{j,3}} \end{bmatrix}^\top \quad (28)$$

$$R_j = \begin{bmatrix} R_{j,1}^\top \\ R_{j,2}^\top \\ R_{j,3}^\top \end{bmatrix}, T_j = \begin{bmatrix} T_{j,1} \\ T_{j,2} \\ T_{j,3} \end{bmatrix} \quad (29)$$

Points whose error residuals are less than a predefined error threshold can be regarded as inliers.

This process is repeated for a number of times until the inlier number reach a predefined inlier threshold. T_i that reach this threshold are regarded as the final solution.

(5) Refine translations T_i using Levenberg-Marquardt method

In Section 4, we have estimated translations T_i robustly using DLT+RANSAC, which is a linear method. However, as shown in (27)-(29), the perspective projection error is nonlinear. Thus, our current estimation is a good starting point, but not yet optimal.

LM (Levenberg-Marquardt) is an algorithm that directly minimize the projection error:

$$T_i = \underset{T_i}{\text{argmin}} \sum_j \|\mathbf{x}_j - \text{projection}_j\|^2 \quad (30)$$

According to LM formulae, T_i is updated by:

$$T_i \leftarrow T_i + \Delta T_i \quad (31)$$

$$\Delta T_i = -(J^\top J + \mu I)^{-1} J^\top r \quad (32)$$

$$r = \begin{bmatrix} r_1 \\ \vdots \\ r_j \\ \vdots \\ r_N \end{bmatrix}, r_j = \begin{bmatrix} \mathbf{x}_{j,1} - \frac{R_{j,1}^\top \mathbf{x}_j + T_{j,1}}{R_{j,3}^\top \mathbf{x}_j + T_{j,3}} \\ \mathbf{x}_{j,2} - \frac{R_{j,2}^\top \mathbf{x}_j + T_{j,2}}{R_{j,3}^\top \mathbf{x}_j + T_{j,3}} \end{bmatrix} \quad (33)$$

$$J = \frac{\partial r}{\partial T} = \begin{bmatrix} J_1 \\ \vdots \\ J_j \\ \vdots \\ J_N \end{bmatrix}, J_j = \begin{bmatrix} -\frac{1}{R_{j,3}^\top \mathbf{X}_j + T_{j,3}} & 0 & \frac{R_{j,1}^\top \mathbf{X}_j + T_{j,1}}{(R_{j,3}^\top \mathbf{X}_j + T_{j,3})^2} \\ 0 & -\frac{1}{R_{j,3}^\top \mathbf{X}_j + T_{j,3}} & \frac{R_{j,1}^\top \mathbf{X}_j + T_{j,1}}{(R_{j,3}^\top \mathbf{X}_j + T_{j,3})^2} \end{bmatrix} \quad (34)$$

As for the damping factor μ :

$$\mu \leftarrow \begin{cases} 10\mu, & \text{if } r \text{ increases} \\ \frac{1}{10}\mu, & \text{if } r \text{ decreases} \end{cases} \quad (35)$$

LM stops updating T_i when ΔT is smaller than a predefined threshold.

(6) Final Triangulations

All camera matrices $P_i = [R_i|T_i]$ are now estimated. To reconstruct the scene, adjacent SIFT feature matches $\mathbf{x}_i \sim \mathbf{x}_{i+1}$ are used to estimate scene points using DLT algorithm, which is shown in (14)-(17).

Specifically, residuals as shown in (27)-(29) are used to evaluate the estimation. Only scene points whose residuals are smaller than a predefined threshold are regarded as inliers and finally saved.

(7) Experiments

The SfM algorithm shown above is coded in MATLAB, and tested by the provided 9 datasets. According to the experiments, it can be concluded that SfM algorithm can robustly reconstruct the scenes and cameras' positions from pictures taken from different directions. Specifically, each dataset result is shown and analyzed below.

(7.1) Experiment on Dataset 1

As shown in figure 1, dataset 1 includes 2 images of the Skansen Kronan redoubt.

It can be concluded that our SfM algorithm performs excellent on this dataset, as the reconstructed point cloud has shown the clear structure of the walls and the windows of the redoubt. Besides, estimated camera positions are also reasonable.

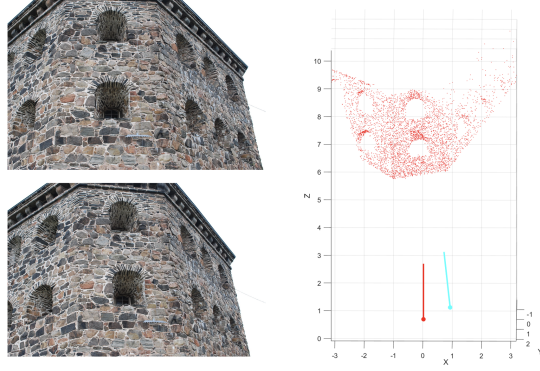


Figure 1: The figure has shown the original images of dataset 1 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

(7.2) Experiment on Dataset 2

As shown in figure 2, dataset 2 includes 9 images of a building's corner.

It can be concluded that our SfM algorithm performs excellent on this dataset, as the reconstructed point cloud has shown the clear structure of the walls and the sunken window. Different point colors denote different cameras' contribution, and it can be noticed that these points overlaps well respectively. Besides, estimated camera positions are also reasonable. They are placed side by side, facing the building's corner. It denotes that these pictures are continuously taken horizontally.

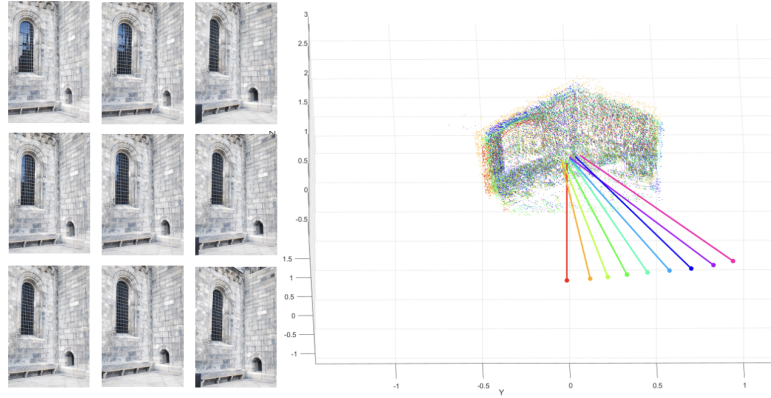


Figure 2: The figure has shown the original images of dataset 2 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

(7.3) Experiment on Dataset 3

As shown in figure 3, dataset 3 includes 12 images of a Arch-shaped gate.

It can be concluded that our SfM algorithm performs excellent on this dataset, as the reconstructed point cloud has shown the clear arch structure of the gate. Different point colors denote different cameras' contribution, and it can be noticed that these points overlaps well respectively. Although because of the SIFT feature outliers, there are drifts between the estimations, our algorithm has tried to estimate robustly.

Besides, estimated camera positions are also reasonable. They are placed side by side, facing the gate. It denotes that these pictures are continuously taken horizontally.

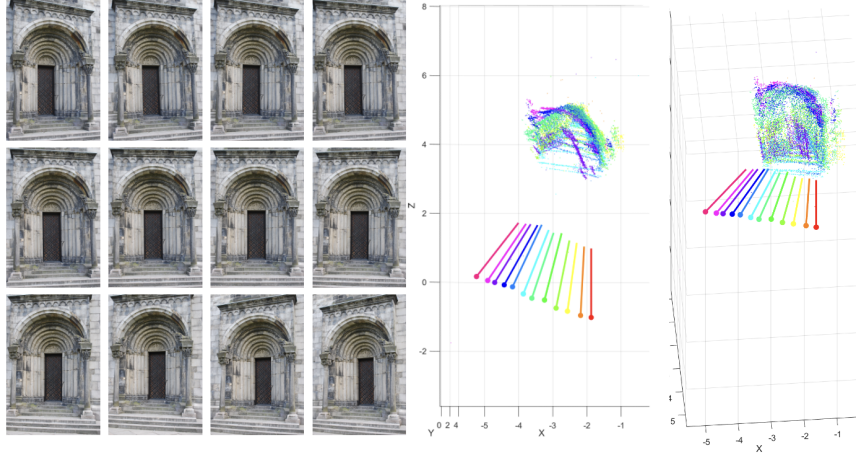


Figure 3: The figure has shown the original images of dataset 3 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

(7.4) Experiment on Dataset 4

As shown in figure 4, dataset 4 includes 14 images of a round fountain.

It can be concluded that our SfM algorithm performs excellent on this dataset, as the reconstructed point cloud has shown the clear arch structure of the fountain. Different point colors denote different cameras' contribution, and it can be noticed that these points overlaps well respectively. However, since these pictures are taken in dark light environment, SIFT feature outlier increases. It can be noticed that there are drifts between the estimations, and our algorithm has tried to estimate robustly.

Besides, estimated camera positions are also reasonable. They are placed side by side, facing the fountain. It denotes that these pictures are continuously taken horizontally.

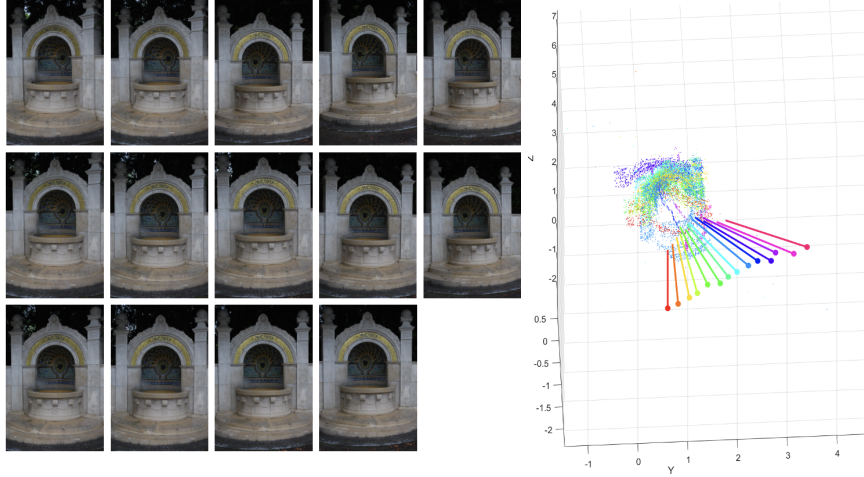


Figure 4: The figure has shown the original images of dataset 4 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

(7.5) Experiment on Dataset 5

As shown in figure 5, dataset 5 includes 10 images of a buddha statue.

Compared to previous datasets, it can be concluded that our SfM algorithm performs normal on this dataset. The shape of the statue can be distinguished independently from each color's camera, however, these points are poorly overlapped respectively. Since these pictures have similar golden SIFT features, there are a considerable amount of outliers among the feature matches. These outliers lead to disturbances, and impact the robustness of our SfM algorithm.

Despite drifts arise while estimating the rotations and the translations, the estimated camera positions are still reasonable. They are placed side by side, facing the statue. It denotes that these pictures are continuously taken horizontally.

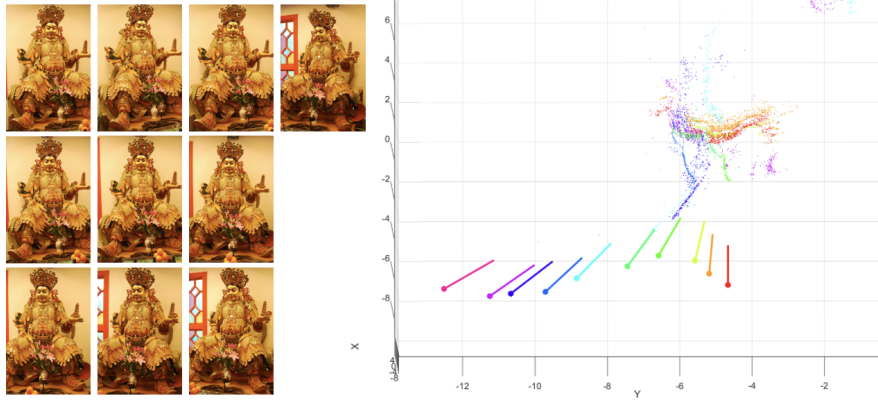


Figure 5: The figure has shown the original images of dataset 5 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

(7.6) Experiment on Dataset 6

As shown in figure 6, dataset 6 includes 8 images of a statue on the wall.

Compared to previous datasets, it can be concluded that our SfM algorithm performs limited on this dataset. The estimated walls from different color's cameras are overlapped, but the statue is hard to be distinguished. This is because of the SIFT outlier matches, which leads to disturbances and impact the robustness of our SfM algorithm.

Moreover, Since the statue is embedded on the wall, and the wall is a dominant plane, homography estimation performs better than the 8-point essential matrix estimation. However, although the cameras are still placed side by side facing the statue, it can be noticed that camera positions estimated from homography might be opposite. This is also caused by the disturbances.

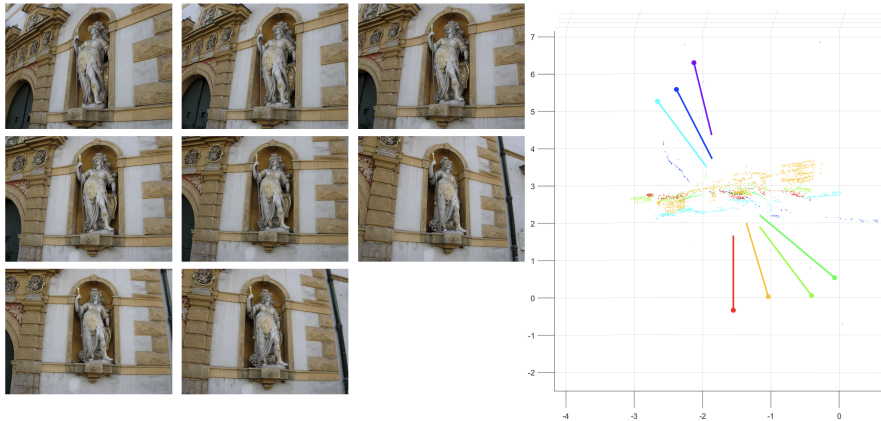


Figure 6: The figure has shown the original images of dataset 6 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

(7.7) Experiment on Dataset 7, 8 and 9

Our SfM algorithm is also tested on dataset 7, 8 and 9. They are respectively 7 images of a building, 12 images of a sculpture and 11 images of a toy dinosaur on a book.

Unfortunately, it can be noticed that the algorithm is limited on these three datasets. Although RANSAC is already applied for outlier rejection, and both the essential matrix and homography were tested for estimating camera motions depending on different scenes, the reconstruction still failed.

- Uneven feature distribution: Large textureless or repetitive regions caused SIFT matches to cluster, leading to unstable motion estimation.
- Small baseline: Limited viewpoint change resulted in low triangulation accuracy, collapsing points to a plane.
- RANSAC limitations: High outlier ratio or poor threshold settings still allowed false matches to dominate.

While the classical SfM pipeline with SIFT, RANSAC, and essential/homography estimation provides a solid baseline, it also shows clear limitations in challenging scenarios. Compared to hand-crafted features, learned descriptors can provide more robust and evenly distributed correspondences. In addition, more advanced outlier rejection schemes can reduce drift and degeneracy better. These directions highlight the potential for extending the current approach with state-of-the-art techniques for more reliable performance.

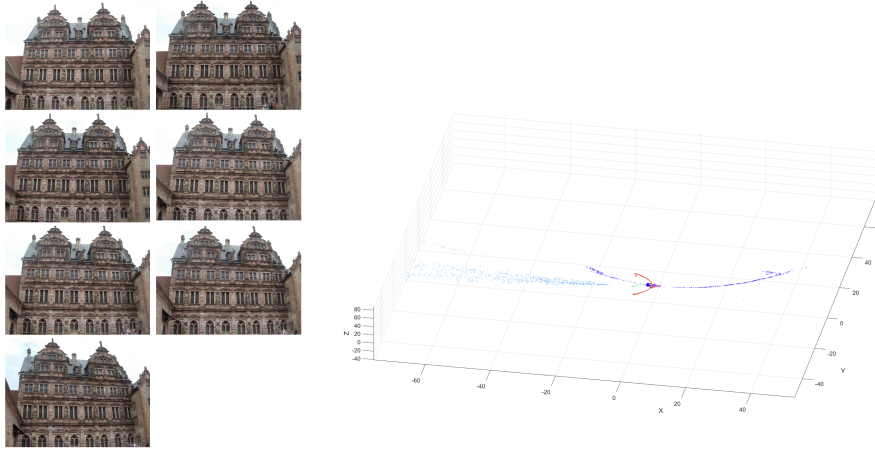


Figure 7: The figure has shown the original images of dataset 7 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

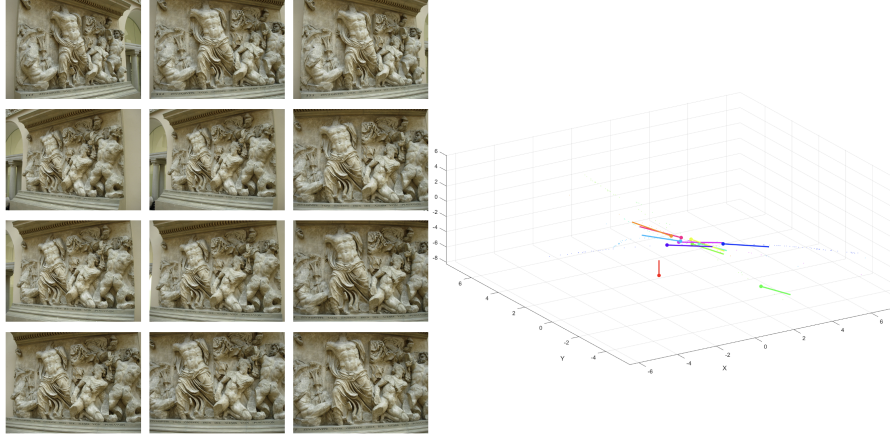


Figure 8: The figure has shown the original images of dataset 8 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).

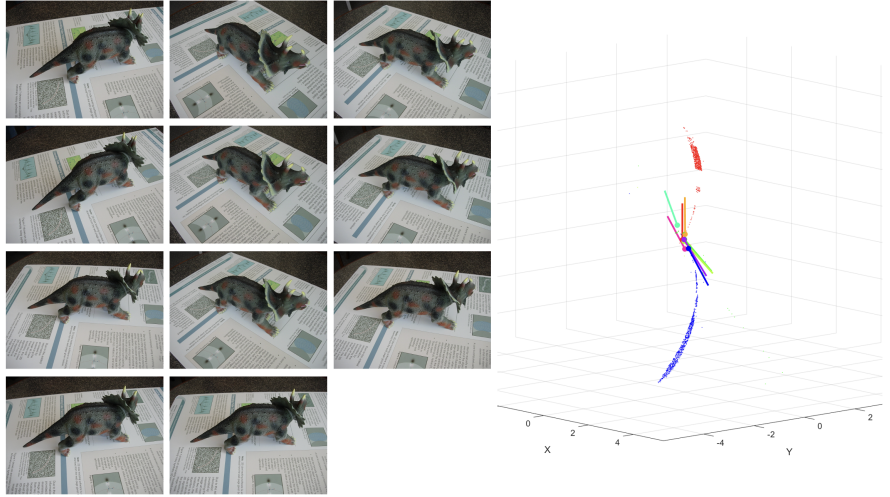


Figure 9: The figure has shown the original images of dataset 9 (left), and the reconstructed point cloud with estimated cameras by our SfM algorithm (right).